

Nhằm đáp ứng các yêu cầu chức năng và phi chức năng đã được phân tích tại Chương 2, hệ thống được xây dựng dựa trên nền tảng của các chuẩn lý thuyết và công nghệ hiện đại. Chương này trình bày hai nhóm nội dung chính: các nền tảng lý thuyết về mô hình phân quyền, giao thức xác thực và JWT; và lựa chọn công nghệ triển khai kèm phân tích so sánh với các phương án thay thế.

0.1 Nền tảng lý thuyết

0.1.1 Mô hình phân quyền dựa trên vai trò (RBAC)

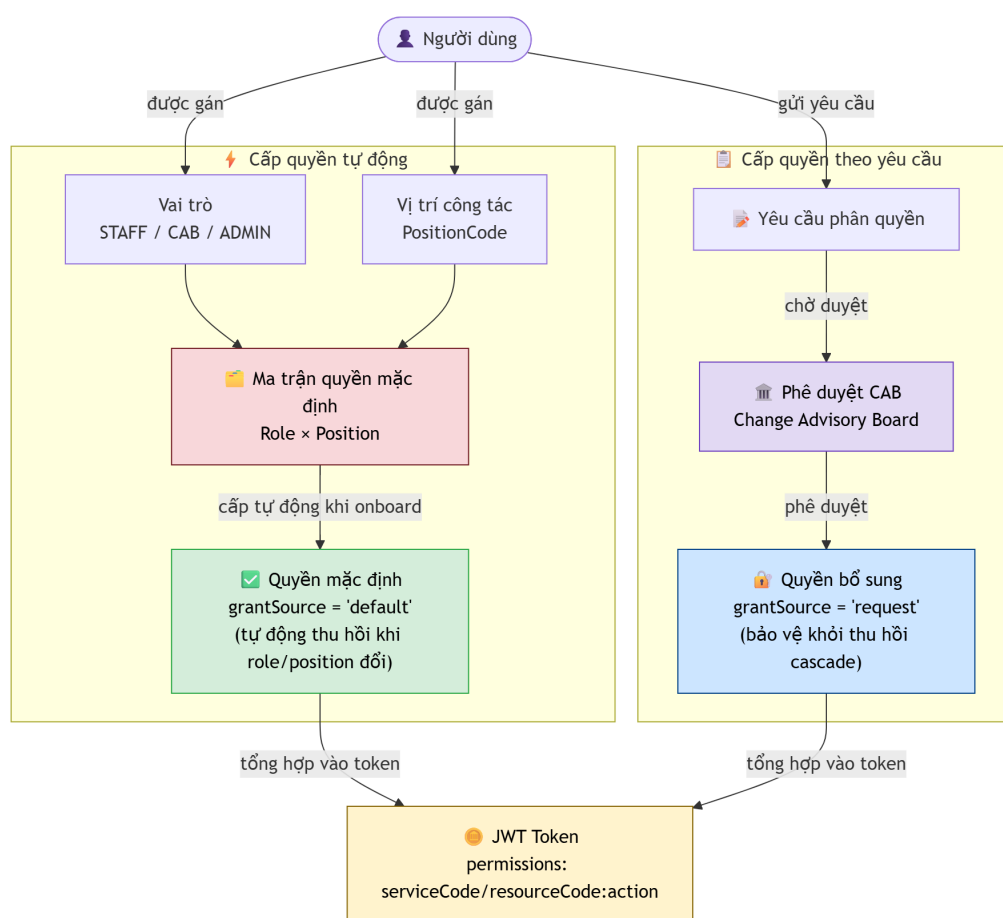
Phân quyền dựa trên vai trò (Role-Based Access Control — RBAC) là mô hình kiểm soát truy cập trong đó quyền hạn không được gán trực tiếp cho người dùng mà được gán cho vai trò (role), và người dùng được gán một hoặc nhiều vai trò phù hợp với vị trí công tác. Mô hình này được Sandhu và cộng sự hệ thống hóa lần đầu vào năm 1996 trong bài báo đăng trên tạp chí IEEE Computer **sandhu1996rbac**, sau đó được chuẩn hóa thành tiêu chuẩn NIST chính thức vào năm 2001 **ferraiolo2001rbac**. Bốn thành phần cốt lõi của RBAC gồm: tập hợp người dùng (Users), tập hợp vai trò (Roles), tập hợp quyền hạn (Permissions) và các phiên làm việc (Sessions). Vai trò hoạt động như một cầu nối trung gian: thay vì quản lý hàng nghìn cặp gán quyền cho từng cá nhân, chúng ta chỉ cần quản lý số lượng vai trò hữu hạn và các quyền gắn với mỗi vai trò — khi nhân viên thay đổi vị trí, chỉ cần điều chỉnh vai trò thay vì xem xét lại từng quyền riêng lẻ.

So với hai mô hình phổ biến khác, RBAC có ưu điểm rõ rệt trong bài toán phân quyền nội bộ ngân hàng. Danh sách kiểm soát truy cập (Access Control List — ACL) gán quyền trực tiếp theo cặp người dùng — hành động cho từng tài nguyên cụ thể, dẫn đến bùng nổ số lượng quy tắc và khó bảo trì khi tổ chức có hàng trăm nhân viên và hàng chục tài nguyên. Kiểm soát truy cập dựa trên thuộc tính (Attribute-Based Access Control — ABAC) cho phép biểu diễn chính sách phức tạp dựa trên nhiều thuộc tính của người dùng, tài nguyên và môi trường, nhưng đòi hỏi hệ thống đánh giá chính sách (policy engine) phức tạp và khó kiểm toán. Trong ngân hàng, nhân viên có vai trò công việc rõ ràng và tương đối ổn định, quyền hạn gắn liền với vị trí tổ chức — đây chính là bài toán RBAC điển hình.

Trong phạm vi đề tài, mô hình RBAC được mở rộng theo hai chiều để đáp ứng yêu cầu vận hành thực tế. Chiều thứ nhất là cơ chế cấp quyền mặc định tự động thông qua ma trận **Role × Position**: khi một nhân viên được gán vai trò và vị trí công tác cụ thể, hệ thống tự động tra cứu ma trận quyền mặc định và cấp toàn bộ quyền phù hợp mà không cần quản trị viên thao tác thủ công. Chiều thứ hai là cơ chế quyền bổ sung qua quy trình phê duyệt. Khi nhân viên có nhu cầu thực hiện một hành động nằm ngoài phạm vi quyền theo vai trò của mình, họ gửi yêu cầu

phân quyền và chờ xét duyệt bởi hội đồng CAB (Change Advisory Board). Sự phân biệt này đảm bảo tính linh hoạt trong cấp quyền đặc biệt mà không làm mất đi tính kiểm soát của hệ thống.

Trong phạm vi đề tài này, mỗi quyền trong hệ thống được biểu diễn dưới dạng chuỗi có cấu trúc *"serviceCode/resourceCode:action"*. Ví dụ, chuỗi *"iam-service/user:create"* biểu diễn quyền tạo người dùng mới trong dịch vụ IAM, còn *"change-mgmt/change-request:approve"* biểu diễn quyền phê duyệt yêu cầu thay đổi trong hệ thống quản lý thay đổi nội bộ. Định dạng phân cấp ba tầng này cho phép kiểm soát quyền ở mức độ chi tiết đến từng hành động trên từng tài nguyên của từng dịch vụ, đồng thời dễ đọc và dễ kiểm toán.

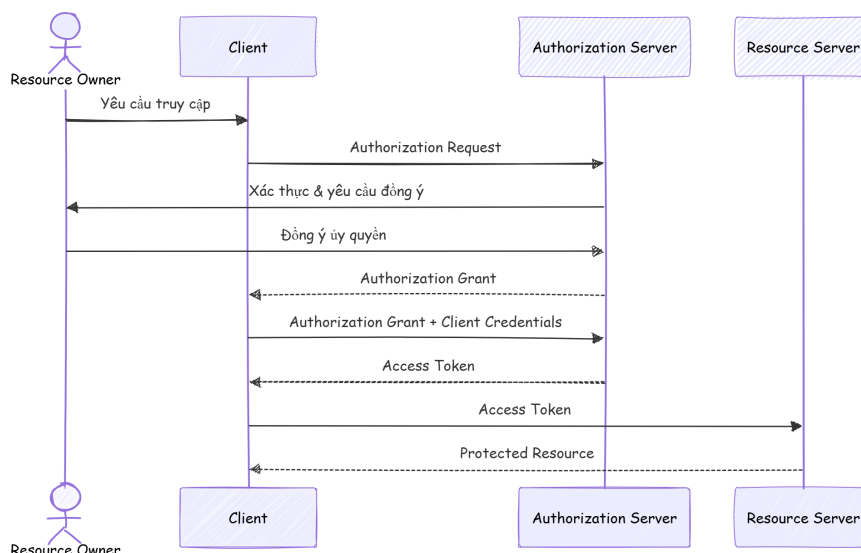


Hình 0.1: Mô hình RBAC mở rộng với ma trận quyền mặc định và quy trình phê duyệt

0.1.2 Chuẩn OAuth 2.0, OpenID Connect và PKCE

OAuth 2.0 **rfc6749** là một framework ủy quyền (authorization framework) mở, cho phép ứng dụng của bên thứ ba truy cập tài nguyên được bảo vệ thay mặt người dùng mà không cần biết thông tin xác thực của họ. OAuth 2.0 định nghĩa bốn vai trò tham gia vào quá trình ủy quyền: Chủ sở hữu tài nguyên (*Resource Owner*), Ứng dụng khách (*Client*), Máy chủ ủy quyền (*Authorization Server*) và Máy chủ

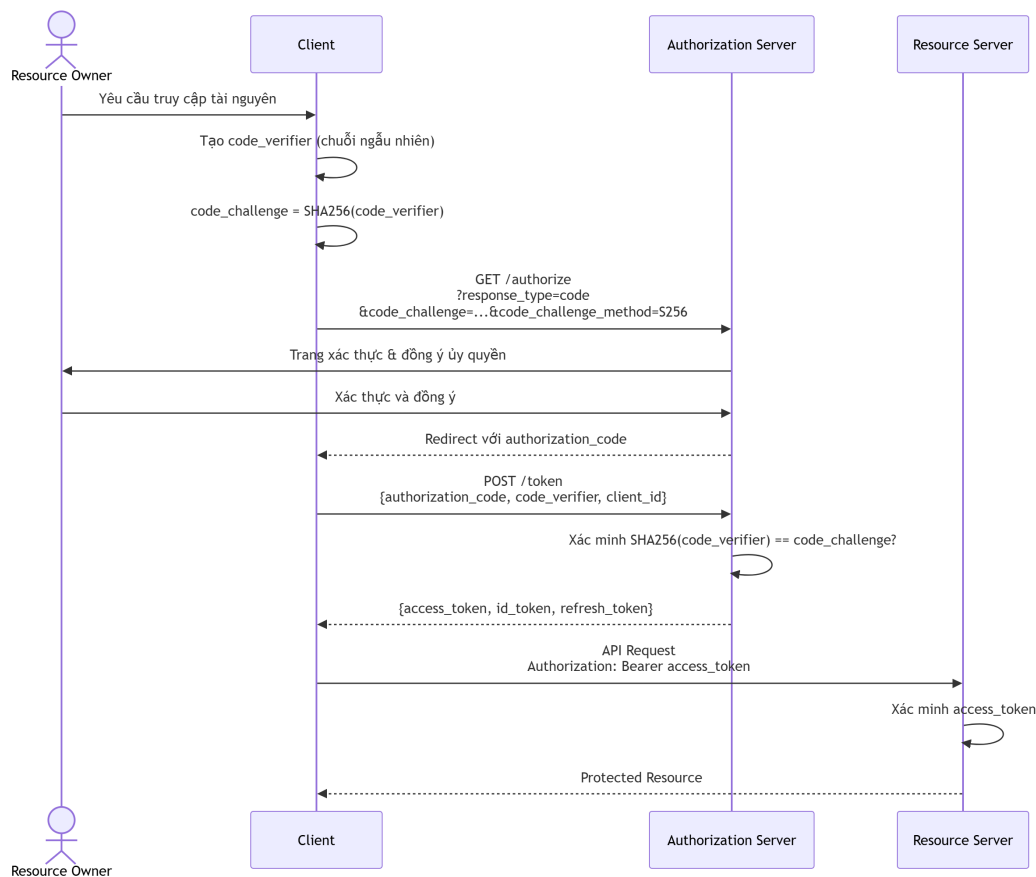
tài nguyên (*Resource Server*).



Hình 0.2: Luồng ủy quyền tổng quát theo chuẩn OAuth 2.0

OpenID Connect (OIDC) **oidc_core_2014** là một lớp giao thức định danh xây dựng trên nền OAuth 2.0, bổ sung khả năng xác thực người dùng (authentication) bên cạnh chức năng ủy quyền (authorization) vốn có. Nếu OAuth 2.0 trả lời câu hỏi “*ứng dụng này được phép làm gì thay mặt người dùng?*”, thì OIDC bổ sung câu trả lời cho câu hỏi “*người dùng này là ai?*”. OIDC giới thiệu thêm ID Token — một JSON Web Token chứa thông tin định danh người dùng — và chuẩn hóa UserInfo endpoint để ứng dụng có thể lấy thêm thông tin hồ sơ. So với SAML (Security Assertion Markup Language) — chuẩn SSO doanh nghiệp truyền thống dựa trên XML — OIDC nhẹ hơn đáng kể, hoàn toàn dựa trên JSON và REST, dễ tích hợp với ứng dụng SPA và các API service hiện đại trong kiến trúc microservices.

Để bảo vệ Authorization Code Flow khi được sử dụng bởi các ứng dụng công khai (public client) như Angular SPA — những ứng dụng chạy hoàn toàn trên trình duyệt và không thể lưu trữ an toàn *client_secret* — RFC 7636 **rfc7636** giới thiệu cơ chế PKCE (Proof Key for Code Exchange). Cơ chế hoạt động theo nguyên tắc cam kết-tiết lộ: khi bắt đầu luồng xác thực, ứng dụng tạo ngẫu nhiên một chuỗi *code_verifier* và tính toán *code_challenge* bằng cách băm chuỗi đó, sau đó gửi *code_challenge* lên Authorization Server; khi đổi authorization code lấy token, ứng dụng gửi kèm *code_verifier* gốc để server xác minh tính khớp. Nhờ đó, ngay cả khi authorization code bị đánh cắp trong quá trình truyền tải, kẻ tấn công cũng không thể đổi lấy token vì không có *code_verifier* tương ứng.



Hình 0.3: Luồng xác thực Authorization Code kết hợp PKCE

0.1.3 JSON Web Token (JWT)

JSON Web Token (RFC 7519 **rfc7519**) là chuẩn mở định nghĩa cách biểu diễn claims — tập hợp thông tin về một thực thể — giữa các bên dưới dạng đối tượng JSON nhỏ gọn, có thể được ký số và xác minh. Một JWT gồm ba phần phân cách bởi dấu chấm: Header (chứa loại token và thuật toán ký), Payload (chứa các claim như subject, thời gian phát hành, thời gian hết hạn và các thông tin tùy chỉnh), và Signature (chữ ký số đảm bảo tính toàn vẹn của hai phần đầu). Đặc tính nổi bật của JWT là khả năng xác minh stateless: bất kỳ bên nào có khóa xác minh đều có thể kiểm tra tính hợp lệ của token mà không cần truy vấn lại bên phát hành.

Bên cạnh các claim chuẩn, JWT payload cho phép nhúng thêm các claim tùy chỉnh. Trong bối cảnh kiểm soát truy cập, các quyền hạn của người dùng có thể được nhúng trực tiếp vào payload tại thời điểm phát hành token, cho phép Resource Server kiểm tra quyền truy cập chỉ bằng cách phân tích nội dung token mà không phát sinh truy vấn thêm — đây là ưu điểm quan trọng giúp giảm độ trễ trong kiến trúc phân tán.

JWT hỗ trợ hai nhóm thuật toán ký: đối xứng (như HS256 — dùng chung một khóa bí mật) và bất đối xứng (như RS256, ES256 — dùng cặp khóa công khai/riêng

tư). Với thuật toán bất đối xứng, khóa riêng chỉ nằm tại Authorization Server để ký token, còn khóa công khai được phân phối qua JWKS endpoint để mọi Resource Server có thể tự xác minh mà không cần liên hệ trực tiếp với Authorization Server. Tuy nhiên, bản chất stateless của JWT đặt ra một đánh đổi quan trọng: token đã phát hành vẫn hợp lệ cho đến khi hết hạn tự nhiên ngay cả khi quyền hạn người dùng đã thay đổi — đây là thách thức thu hồi token (token revocation) mà mọi hệ thống sử dụng JWT đều phải cân nhắc giải quyết.

0.2 Công nghệ sử dụng

Lựa chọn công nghệ được đánh giá dựa trên ba tiêu chí: mức độ trưởng thành của hệ sinh thái, sự phù hợp với yêu cầu bảo mật trong môi trường tài chính ngân hàng, và khả năng tích hợp đồng nhất giữa các thành phần trong kiến trúc đã chọn.

0.2.1 Ngôn ngữ lập trình và Framework: Java và Spring Boot

Java là ngôn ngữ lập trình hướng đối tượng, kiểu tĩnh mạnh, đã được kiểm chứng trong môi trường enterprise qua nhiều thập kỷ. Theo Stack Overflow Developer Survey 2025 với hơn 49.000 lập trình viên từ 177 quốc gia tham gia khảo sát, Java nằm trong số các ngôn ngữ được lập trình viên chuyên nghiệp sử dụng rộng rãi nhất trên toàn cầu **stackoverflow_survey_2025**. Spring Boot là framework hàng đầu trong hệ sinh thái Java: theo JetBrains State of Developer Ecosystem 2025 với 24.534 developer từ 194 quốc gia tham gia, Spring Framework được 65% Java developer sử dụng, dẫn đầu tuyệt đối so với mọi framework cạnh tranh khác **jetbrains_deveco_2025**. Bảng 1 đánh giá Java/Spring Boot so với các lựa chọn thay thế tiêu biểu trong bối cảnh xây dựng hệ thống IAM:

Bảng 1: So sánh các ngôn ngữ và framework cho hệ thống IAM

Ngôn ngữ / Framework	Điểm mạnh	Hạn chế đối với bài toán IAM
Go / Gin	Khởi động nhanh, low-latency, nhỏ gọn	Thiếu thư viện OAuth2 Authorization Server, ORM, Kafka tích hợp sẵn ở mức production-ready
Python / FastAPI	Cú pháp ngắn gọn, phổ biến cho AI/data	Thiếu type-safety nghiêm ngặt; hệ sinh thái enterprise security chưa trưởng thành
C++	Hiệu năng phần cứng tối đa	Overhead phát triển và bảo trì quá lớn cho nghiệp vụ IAM phức tạp
Java / Spring Boot	Hệ sinh thái enterprise trưởng thành nhất	Thời gian khởi động chậm hơn Go/Quarkus (có thể tối ưu bằng Docker jlink)

Spring Boot mang lại bộ công cụ sẵn có hoàn chỉnh cho toàn bộ yêu cầu kỹ

thuật của hệ thống trong một hệ sinh thái nhất quán: Spring Authorization Server cho việc xây dựng OAuth 2.0 Authorization Server chuẩn RFC, Spring Security cho bảo vệ Resource Server và xác minh JWT, Spring Cloud Gateway với WebFlux cho reactive API Gateway, Spring Data JPA cho tích hợp ORM với Oracle Database, và Spring Kafka cho producer/consumer Kafka. Sự nhất quán trong cấu hình, dependency injection và xử lý lỗi giữa các thành phần Spring giúp giảm đáng kể chi phí tích hợp so với việc ghép nối các thư viện từ các hệ sinh thái khác nhau.

0.2.2 Hệ quản trị cơ sở dữ liệu: Oracle Database

Oracle Database là hệ quản trị cơ sở dữ liệu quan hệ đã được kiểm chứng trong môi trường tài chính ngân hàng trên toàn thế giới. Các đặc tính kỹ thuật của Oracle đáp ứng trực tiếp các yêu cầu của hệ thống: đảm bảo ACID (Atomicity, Consistency, Isolation, Durability) tuyệt đối cho mọi thao tác thay đổi quyền hạn và vòng đời người dùng; Stored Procedure PL/SQL cho phép đóng gói logic nghiệp vụ phức tạp tại tầng cơ sở dữ liệu. Row-Level Security kiểm soát truy cập ở cấp hàng dữ liệu, phù hợp với yêu cầu audit và bảo mật dữ liệu nhạy cảm trong ngân hàng.

0.2.3 Message Broker: Apache Kafka

Apache Kafka là nền tảng message streaming phân tán, được phát triển ban đầu tại LinkedIn và hiện là dự án mã nguồn mở thuộc Apache Software Foundation. Không giống các message queue truyền thống vốn xóa message sau khi consumer đã đọc, Kafka lưu trữ message trong log bất biến có thể replay được với thời gian lưu trữ cấu hình linh hoạt — đặc tính quan trọng cho yêu cầu audit trail trong môi trường tài chính. Theo Confluent Data Streaming Report 2025 với 4.175 IT leader tham gia khảo sát, hơn 150.000 tổ chức trên toàn cầu đang sử dụng Apache Kafka và 89% IT leader coi nền tảng data streaming là thiết yếu cho mục tiêu dữ liệu của tổ chức **confluent_streaming_2025**. Bảng 2 so sánh Kafka với RabbitMQ — message broker phổ biến trong các hệ thống doanh nghiệp:

Bảng 2: So sánh Apache Kafka và RabbitMQ

Tiêu chí	RabbitMQ	Apache Kafka
Mô hình lưu trữ	Queue — xóa message sau khi consume	Log bất biến — giữ lại, có thể replay
Throughput	Vừa phải	Rất cao — phù hợp event streaming
Fan-out	Cần cấu hình exchange thủ công	Tự nhiên — nhiều consumer group độc lập
Audit trail	Không có sẵn	Có — log retention cấu hình được
Phù hợp khi	Task queue đơn giản, định tuyến phức tạp	Event streaming, audit log, fan-out

Kafka được lựa chọn vì ba đặc điểm phù hợp trực tiếp với yêu cầu hệ thống. Thứ nhất, khả năng fan-out tự nhiên cho phép một sự kiện thu hồi quyền được xử lý đồng thời bởi nhiều consumer — chấm dứt phiên làm việc và gửi thông báo — mà không cần publisher quản lý danh sách subscriber. Thứ hai, cơ chế lưu trữ log đảm bảo không mất sự kiện khi consumer tạm thời gặp sự cố, consumer sẽ tự động xử lý lại khi phục hồi. Thứ ba, Kafka log retention phục vụ yêu cầu audit trail: mọi sự kiện thay đổi quyền hạn đều được lưu lại và có thể replay để kiểm toán sau này — yêu cầu thiết yếu trong ngân hàng nơi mọi thay đổi truy cập phải có dấu vết ghi chép.

0.3 Kết luận chương

Chương này đã trình bày nền tảng lý thuyết và công nghệ làm cơ sở xây dựng hệ thống phân quyền. Về lý thuyết, mô hình RBAC mở rộng kết hợp ma trận quyền mặc định theo Role và Position với quy trình phê duyệt CAB tạo ra sự cân bằng giữa tự động hóa và kiểm soát chặt chẽ; chuẩn OAuth 2.0, OIDC và PKCE đảm bảo luồng xác thực và ủy quyền tuân theo các chuẩn mực bảo mật quốc tế; JWT cho phép truyền tải quyền hạn stateless và hiệu quả, với bài toán thu hồi token được giải quyết ở tầng thiết kế hệ thống. Về công nghệ, bộ Java/Spring Boot, Oracle Database và Apache Kafka là lựa chọn phù hợp với môi trường tài chính ngân hàng nhờ hệ sinh thái trưởng thành, tính bảo mật được kiểm chứng qua thực tiễn và khả năng tích hợp tự nhiên trong cùng một kiến trúc. Trên cơ sở nền tảng này, Chương 4 sẽ trình bày lựa chọn kiến trúc hệ thống, thiết kế chi tiết các thành phần và kết quả triển khai thực tế.